Exercise Session 2: Classical Foundations Edges, Voting, Shape

Automatic Image Analysis

Exercise Session 2

Classical foundations for image analysis: edges, voting, and shape descriptors.

- Classical tools for image analysis and their connection to project topics

Computer Vision and Remote Sensing Group, TU Berlin
Summer Semester 2026

Contact and consultation

Contact

Name: Annamalai Lakshmanan

Email: annamalai.lakshmanan@tu-berlin.de

Office: Room 6.048, MAR Building

Consultation hours

5pm - 6 pm Tuesdays unless otherwise announced. You can send an email to schedule a different time if needed.

Exercise 2: why these tools matter

Classical components

- Edges and gradients support leaf contours and local features.
- Hough-style voting explains how local evidence can become a global hypothesis.
- HOG turns gradients into a classifier-ready vector.

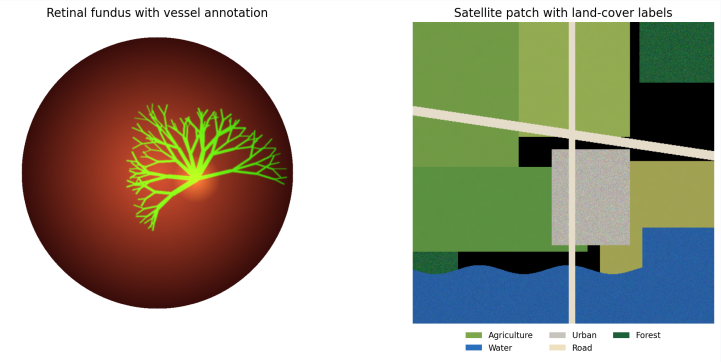
Project connections

- Topic A: leaf shape and boundary descriptors.
- Topic C: repeatable keypoints and descriptors.
- Topic D: HOG baseline before DINO features.
- Topic E: gradients and motion as temporal evidence.

Flow of the session

First we discuss the image-analysis ideas. Then we move to datasets, notebooks, packages, CUDA, scikit-learn, and PyTorch.

What information does a pixel encode?



Opening question

What information does a pixel encode?

- Intensity
- Color
- Gradient
- Local contrast

From pixel values to meaning

Two levels of interpretation

The same pixel can be read as a measurement or as evidence for a semantic concept.

Low-level view

- Numeric brightness or color.
- Local change in space.
- Texture and contrast cues.

High-level view

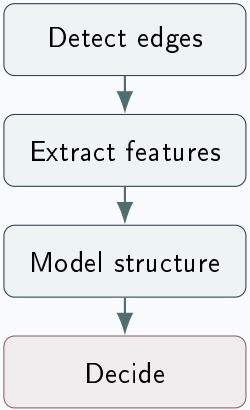
- Road or building edge.
- Leaf contour or venation.
- Region, class, or object.

The classical paradigm as a pipeline

Core idea

Classical computer vision builds meaning through explicit, hand-engineered decisions.

- Start from measurable local evidence.
- Move to features or hypotheses.
- End with a decision.



Why this pipeline matters

Tension of the course

We constantly move between low-level signal and high-level interpretation.

- Edge detectors simplify raw measurements.
- Features summarize repeated local structure.
- Geometric models explain larger patterns.
- Final decisions translate structure into meaning.

Take-away

Classical methods are transparent because every step specifies what evidence should count.

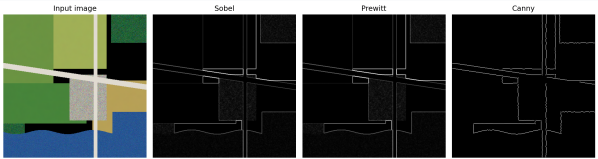
Edges as the first abstraction

Gradient model

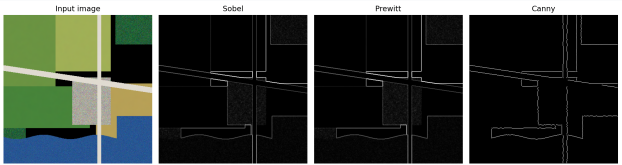
$$\nabla I(x,y) = \begin{bmatrix} G_x \\ G_y \end{bmatrix}, \qquad \|\nabla I\| = \sqrt{G_x^2 + G_y^2}$$

$$\theta = \text{atan2}(G_y, G_x)$$

- Sobel and Prewitt approximate spatial derivatives.
- Magnitude highlights discontinuities.
- Orientation gives edge direction.



What edges give us



Interpretation

An edge detector suppresses interior texture and preserves transitions that may correspond to object boundaries.

- Boundaries become easier to localize.
- Smooth regions become less important.
- Geometry starts to emerge from raw intensity.

Why Canny remains the classical reference

Key idea

Compared with a single threshold, Canny produces thinner and more stable contours.

Step 1

Non-maximum suppression keeps only local maxima along the gradient direction.

Result

Thick gradient responses collapse into thin contour candidates.

Canny: linking weak and strong edges

Step 2

Hysteresis thresholding keeps weak edges only when they connect to strong ones.

- Removes isolated noise responses.
- Preserves longer, coherent boundaries.

Why it matters

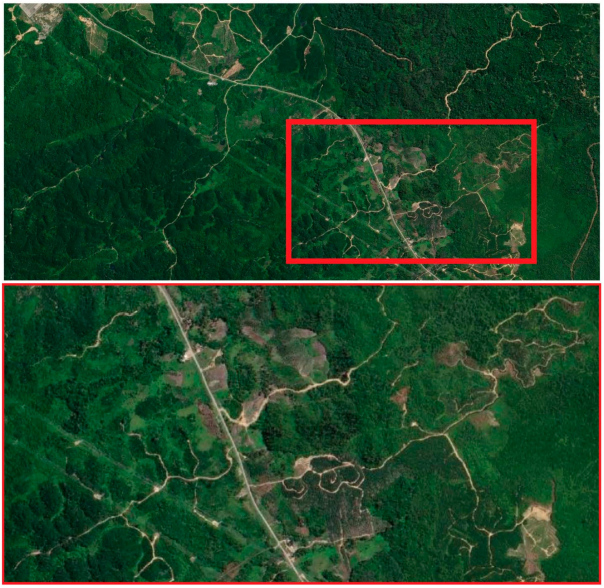
Canny mixes local evidence with short-range structural consistency.

Canny on low-contrast satellite imagery

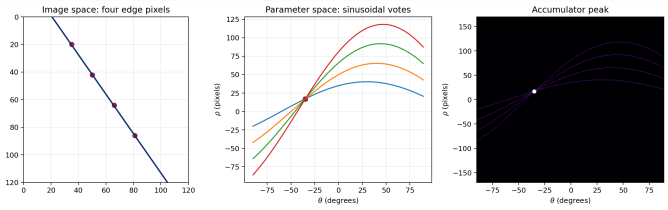
Discussion

What happens on a satellite image with low-contrast boundaries, haze, shadows, or gradual land-cover transitions?

- Missed boundaries when gradients stay below threshold.
- Fragmented contours when texture dominates geometry.
- Parameter sensitivity in smoothing and thresholds.
- Strong dependence on image formation conditions.



The Hough principle: from edges to votes



- Each edge pixel votes for all lines consistent with that observation.
- In image space, a line is a set of collinear pixels.
- Collinear points reinforce one shared hypothesis.

The Hough principle: parameter space view

Line parameterization

$$\rho = x \cos\theta + y \sin\theta$$

- Each image pixel becomes a sinusoid in parameter space.
- Intersections correspond to shared line hypotheses.

Accumulator intuition

Peaks appear when many local observations agree on the same global geometry.

From line voting to generalized Hough

Generalization

The generalized Hough transform replaces a fixed parametric family with an **R-table**. Edge orientation then votes for possible positions of a known reference point.

- The core principle stays the same: local evidence accumulates into a global hypothesis.
- The representation becomes flexible enough for known shapes.
- This is the direct conceptual bridge from line detection to Project 2.

Feature Matching

For each point, how to **match** its **corresponding point** in the other image?

- **Brute-force Matching: compare each feature** descriptor **of Image 1 against** the descriptor of **each feature in Image 2** and assign as correspondence the feature with **closest descriptor** (e.g., minimum of SSD). If each image contains N features, we need to perform N^2 **comparisons**.

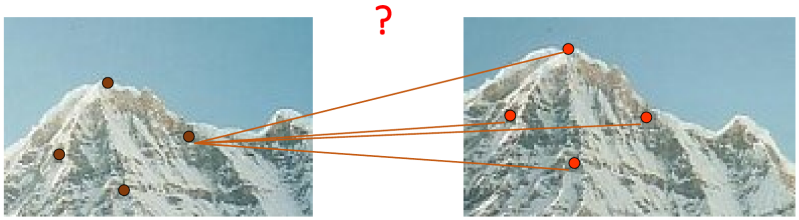


Image 1 Image 2

Shape descriptors: what makes a good feature?

Four Important Things

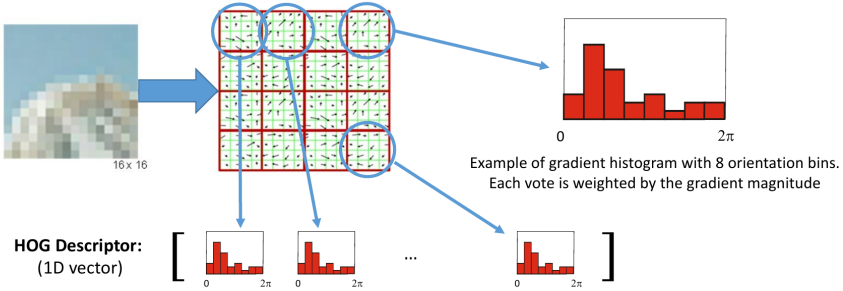
- **Invariance:** translation, rotation, scale, illumination.
- **Discriminability:** separate similar classes reliably.
- **Compactness:** store enough information in a short vector.
- **Computational cost:** remain feasible for large datasets.

Guiding question

A useful descriptor should stay stable when irrelevant image conditions change, but still separate genuinely different structures.

HOG Descriptor (Histogram of Oriented Gradients)

- The patch is divided into a **grid of cells** and for each cell a **histogram of gradient directions weighted by the gradient magnitude** is compiled.
- The HOG descriptor is the **concatenation of these histograms** (used in SIFT)
- Differently from the patch and Census descriptors, HOG has **float values**.



Excerpt from: Vision Algorithms for Mobile Robotics, Prof. Davide Scaramuzza, University of Zurich

HOG as a descriptor

Main idea

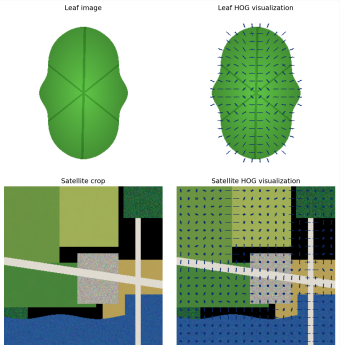
Histogram of Oriented Gradients aggregates local edge directions into a compact shape-and-appearance representation.

- Pool gradients locally.
- Build orientation histograms.
- Concatenate them into one feature vector.

Strength and limitation

Block normalization helps with illumination changes, but standard HOG is not rotation invariant.

HOG: local appearance in practice



- On the leaf example, HOG captures the outer contour and dominant venation pattern.
- On the satellite crop, it emphasizes roads, parcel boundaries, and building edges.

Where HOG fits best

Discussion

For which project would HOG be most useful, and why?

- Topic A: HOG can capture leaf contour and venation as a baseline next to Fourier descriptors and Hu moments.
- Topic D: HOG is the classical baseline before comparing against DINO features.
- Topic C: the same gradient logic motivates local descriptors such as SIFT.
- It is less suitable when rotation, scale change, or temporal order dominates the task.

Now move from ideas to implementation

Implementation goal

By the next working step, each group should be able to load data, run a small baseline, save results, and know which compute environment they are using.

Choose one environment

- Google Colab.
- Kaggle notebooks.
- Local Conda/Miniconda.

Record from day one

- Dataset version and subset.
- Package versions.
- Random seed and train/test split.
- Runtime and compute platform.

Google Colab setup

Start a notebook

1. Open Google Colab.
2. Create a new notebook.
3. Runtime → Change runtime type.
4. Select GPU only if needed.
5. Mount Drive only if the data is stored there.

First cells

```
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
import cv2
DATA = Path("/content/data")
```

Practical note

Colab storage is temporary. Save important figures, features, and notebooks back to Drive or download them.

Kaggle setup

Notebook workflow

1. Open Kaggle and create a notebook.
2. Attach the dataset from the right sidebar.
3. Enable GPU if needed under accelerator settings.
4. Read files from `/kaggle/input/...`.
5. Write outputs to `/kaggle/working/...`.

Local setup with Miniconda

Why Conda

Conda keeps project packages separate from your system Python and makes it easier to reproduce the environment.

Create environment

```
conda create -n aia python=3.11
conda activate aia
python -m pip install -upgrade pip
```

Install packages

```
pip install numpy pandas matplotlib tqdm
pip install opencv-python scikit-image
pip install scikit-learn jupyter
```

Save environment

```
pip freeze > requirements.txt
```

or

```
conda env export > environment.yml
```

Basic terminal commands

Files and folders

```
pwd                                  current folder
ls                                        list files
cd project                             enter folder
mkdir -p data/raw results/figures
du -sh data                          check data size
```

Run and document

```
python script.py
jupyter lab
pip freeze > requirements.txt
python -c "import cv2, sklearn, torch"
python -c "import numpy as np"
```

Rule

If a command changes data, packages, or results, keep it in the project notes or repository.

Recommended project structure

Folder layout

```
project/
  data/raw/
  data/processed/
  notebooks/
  src/
  results/figures/
  results/tables/
```

Use the dataset safely

- Keep downloaded data unchanged in `data/raw/`.
- Write resized images, cached features, and splits to `data/processed/`.
- Save plots and tables under `results/`.
- Use relative paths through `pathlib.Path`.

Reading image datasets

OpenCV

```
img = cv2.imread(str(path))
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
```

Display

```
plt.imshow(img)
plt.axis("off")
plt.savefig("results/figures/example.png")
```

For project data

- Leaf and skin projects: inspect labels and class balance first.
- Matching project: keep image-pair order and homography files together.
- Video project: start with a few clips before decoding all videos.

scikit-learn basics

Train/test split

```
from sklearn.model_selection import
train_test_split
Xtr, Xte, ytr, yte = train_test_split(
  X, y, test_size=0.3, stratify=y,
  random_state=0)
```

Fit and evaluate

```
clf = LinearSVC()
clf.fit(Xtr, ytr)
pred = clf.predict(Xte)
accuracy_score(yte, pred)
confusion_matrix(yte, pred)
```

Classifiers

```
from sklearn.svm import LinearSVC
from sklearn.neighbors import
KNeighborsClassifier
```

Where this appears

Leaf, HOG, SIFT/SuperPoint, DINO, and optical-flow baselines all need consistent splits and metrics.

PyTorch dataset basics

ImageFolder pattern

```
from torchvision import datasets, transforms
tfm = transforms.Compose([...])
ds = datasets.ImageFolder("data/raw", tfm)
loader = DataLoader(ds, batch_size=32)
```

Use when

Your data is arranged as class folders, for example `class_name/image.png`.

Custom Dataset pattern

```
class MyDataset(Dataset):
  def __len__(self):  ...
  def __getitem__(self, i):  ...
```

Use when

Labels live in CSV files, videos need decoding, or each example has multiple files.

CUDA and GPU checks

What CUDA means here

CUDA is NVIDIA's GPU computing platform. You do not need to program CUDA directly; PyTorch uses it for faster feature extraction and training.

PyTorch check

```
import torch
torch.cuda.is_available()
device = "cuda" if torch.cuda.is_available()
else "cpu"
model.to(device)
```

System check

`nvidia-smi`

Shows GPU model, driver, memory use, and running processes.

Fallback

If CUDA is unavailable, run smaller batches, cache features, or use Colab/Kaggle GPU.

First reproducible baseline

Do this before the full project

Run a tiny end-to-end experiment on 5–10 examples from your topic.

- Load examples and labels.
- Visualize raw input and one processed output.
- Extract one simple feature: edge map, HOG, color histogram, or frozen features.
- Train or run one minimal classifier.
- Save figures, parameters, versions, and runtime.

Why

Small tests expose path errors, wrong labels, memory problems, and broken assumptions before the full experiment.

Key message

Before deep features, there is structure

Classical image analysis asks us to specify *what* matters in the image and *how* evidence should accumulate: gradients define edges, votes define geometry, and descriptors define similarity.

- Edges convert intensity changes into candidate boundaries.
- Hough voting converts local evidence into global geometric hypotheses.
- Shape descriptors convert neighborhoods into compact feature vectors.
- Your exercise project will examine exactly where these explicit models succeed and where they break.